

Backend Audit & Prioritized Plan

Prepared 2026-08-19. Scope: `main/backend` (Node/Express + Firebase Admin SDK). This is an internal company tool at ~100-500 daily users, scaling toward 1,000+ — not a large-scale SaaS. Recommendations are sized for that; no microservices/Kafka/Redis/GraphQL are proposed.

Do not start implementation from this doc without confirming priority order — fix P0 first, then P1, etc.

1. Current architecture

- **Stack:** Express 4, Firebase Admin SDK (Firestore + Auth), JWT for app sessions, Nodemailer/SMTP, `express-async-errors` for centralized async error handling. Runs either as a long-lived Node process or as a Vercel serverless function (`server.js` skips `.listen()` when `process.env.VERCEL` is set).
- **Layering:** `routes/*.js` → `authMiddleware` → `roleMiddleware` / `requirePermission` → `controllers/*.js` → Firestore directly. No service/repository layer.
- **Auth:** Firebase Auth owns credentials; login re-verifies password via Identity Toolkit REST, then the backend signs its own JWT (7-day expiry, embeds `id/email/role/full_name/sid`). `authMiddleware` re-checks the account against a 60s-TTL Firestore profile cache and a 30s-TTL session-revocation cache, so role changes/deactivation/force-logout take effect within ~1 minute.
- **Authorization:** three layers — `roleMiddleware` (coarse allow-list), `requirePermission` (granular per-role action matrix in Firestore, currently only wired onto IT asset routes), and ownership checks inline in a few controllers. Roles: `employee`, `hr`, `it`, `coordinator`, `founder`, `superadmin`.
- Session tracking (`sessions` collection) and audit logging (`audit_logs`) are real and wired into every sensitive Super Admin mutation.

2. API inventory (by route file)

- `authRoutes` (`/api/auth`) — register (public), login (public), me, verify-password
- `hrRoutes` / `itRoutes` — complaints CRUD + status/fields update (role-gated), IT also has asset CRUD (+ `requirePermission`)
- `founderRoutes` — merged complaints feed, user management, audit logs, analytics (+CSV export), search, activity timeline, dashboard layout/overview, SLA policies, notification rules, role/action permissions, departments — all superadmin-only except a few any-auth reads
- `securityRoutes` (`/api/founder/security`) — sessions, force-logout, failed logins, account unlock — all superadmin-only
- `approvalRoutes`, `leaveRoutes`, `coordinatorRoutes`, `renderRoutes` — role-gated CRUD + decision endpoints for their respective domains
- `hrDeskRoutes` — send-email + generic CRUD factory over `employees`, `candidates`, `interviews`, `meetings`, `attendance`, `feedback`, `jobs` (hr/founder only)

No automated tests exist anywhere under `main/backend`.

3. Problems, prioritized

P0 — Critical

None found that are new. `docs/login-credentials.pdf` (real employee passwords, untracked) is already flagged in `INFRASTRUCTURE_PLAN.md` — just make sure it's never committed and gets deleted from disk.

P1 — High

#	Problem	Where	Fix
1	List endpoints <code>.limit(200)</code> with no <code>.orderBy()</code> first — Firestore returns an arbitrary 200 docs, not the most recent 200. Once a collection exceeds 200 docs, new tickets/tasks/leave requests can silently vanish from queues.	hrController.js:144,152, itController.js:150,158, founderController.js:62-64, approvalController.js:35, leaveController.js:46, assetController.js:41, renderController.js:7, taskProjectController.js:9,15, hrDeskController.js:34,48	Add <code>.orderBy(dateField, 'desc')</code> before <code>.limit(200)</code> , with composite indexes where Firestore requires them
2	Dashboard/analytics endpoints run fully unbounded <code>.get()</code> across users, hr_complaints, it_complaints, approvals, leave_requests, assets, failed_logins — scales with total historical data, not what's displayed. This is the next likely quota/cost blowup as volume grows (same failure class as the prior fixed incident).	founderController.js:322-328 (computeAnalytics), founderController.js:581-593 (getDashboardOverview)	Bound with date-range filters and/or aggregation instead of full scans
3	In-process caches (authMiddleware.js, sessions.js) assume a long-lived process, but the app is also deployable as Vercel serverless — cold starts get an empty cache, so the quota-fix caching may not hold on that deployment path	authMiddleware.js:16, utils/sessions.js:26	Confirm actual deployment target; if serverless, caching strategy needs rethinking
4	Public, unrestricted self-registration — no auth guard, no company-domain check, no rate limit/CAPTCHA. Anyone can create a real account and log into the portal as employee.	authRoutes.js:7, authController.js:27-62	Gate behind admin approval, or at minimum add rate limiting (see P2 #6) and/or domain allow-list

P2 — Medium

#	Problem	Where	Fix
5	Unescaped user input interpolated into HTML emails — ticket name/department and HR-desk email body go straight into HTML strings	utils/mailer.js:36-39, hrDeskController.js:13	Escape HTML before interpolation
6	No rate limiting anywhere (login has per-account lockout, but no IP throttling on <code>/register</code> or elsewhere) — already flagged in INFRASTRUCTURE_PLAN.md	whole app	Add express-rate-limit

7	Generic error handler returns raw <code>err.message</code> to the client for any unexpected error — minor info disclosure	<code>server.js:52-57</code>	Return a generic message for unexpected errors; keep <code>err.message</code> passthrough for intentionally-thrown <code>{status, message}</code> errors
8	<code>hrDeskController.makeCrud()</code> has no field allow-list (unlike every other controller's <code>EDITABLE_FIELDS</code> pattern) — spreads <code>...req.body</code> into Firestore docs	<code>hrDeskController.js:44-84</code>	Add explicit editable-field lists, matching the rest of the codebase
9	No automated tests — authorization logic (ownership checks, department-routing rules, transactional status updates) has no regression coverage	n/a	A handful of integration tests around <code>authMiddleware/roleMiddleware</code> + the transactional pattern would catch regressions cheaply
10	<code>docs/BACKEND_WORKFLOW.md</code> is stale — says most of the UI isn't wired to the backend, but git history shows Tickets/Approvals/Leave/Assets/Tasks/Rendering/HR are all live now	doc only	Update or delete the stale doc so it isn't mistaken for current truth

P3 — Low

- Password minimum is 6 chars, only enforced on admin reset (`founderController.js:262`); registration relies on Firebase Auth's default. Fine for this scale, just noting the floor.
- `LOCK_THRESHOLD = 5` is a hardcoded constant (`authController.js:10`) — deliberate per its own comment, not a problem.

4. Already fine — do not re-fix

- CORS is properly allow-listed with sane `maxAge`, not an open `cors()`.
- Transactional writes for status/approval linkage (`itController`, `hrController`, `approvalController`) correctly use `db.runTransaction` with re-reads before writes.
- Role escalation is properly locked down — self-registration always yields `employee`, `founder` / `superadmin` can't be self-minted, Super Admin can't self-logout.
- Session revocation / force-logout is real and takes effect quickly via the `sid` + short-TTL cache.
- Account lockout after failed logins works correctly, including reset on success.
- Firestore doc-ID injection guard on asset creation (`assetController.js:11`).
- N+1 avoidance in `enrichWithUserRole` — batches and dedupes user lookups correctly.
- Audit logging covers every sensitive Super Admin mutation.
- `.env` is gitignored; `config/firebase.js` fails gracefully to the Local Emulator Suite.
- CSV export properly escapes against CSV/formula injection.

Next step

Confirm priority order, then implement P1 items first (#1 `orderBy` fix is the smallest diff with the highest real-world impact — silently missing tickets in a queue is a support-breaking bug, not a theoretical one).